POLITECNICO MILANO 1863 | DEPARTMENT OF ELECTRONICS INFORMATION AND BIOENGINEERING

Project Assignment

Embedded and Edge Artificial Intelligence

22.04.2026 | Stefano Staffolani

Project Overview

The Task

Build a TinyML system that estimates **2D positions** of up to 4 people inside a room, using raw UWB radar signals from **1 to 6 radars**.

Goal

- Build a **TinyML system** that estimates **2D positions of up to 4 people** inside a room
- **Input**: raw UWB radar signals (CIR) from a **variable number of radars (1 to 6)**
- **Output**: per-frame [x, y] position estimates for each detected person

Target Hardware

- Model must be deployable on an ESP32-S3 microcontroller via TensorFlow Lite for Microcontrollers

Deliverables & Teams

Teams

- Groups of **2 students**
- Submitted via GitHub Classroom

Deliverables

- `code.py` -> inference script (NumPy + SciPy + TensorFlow only)
- `model.tflite` -> trained TFLite model
- `report.pdf` -> project report
- All training notebooks, scripts, and experiments

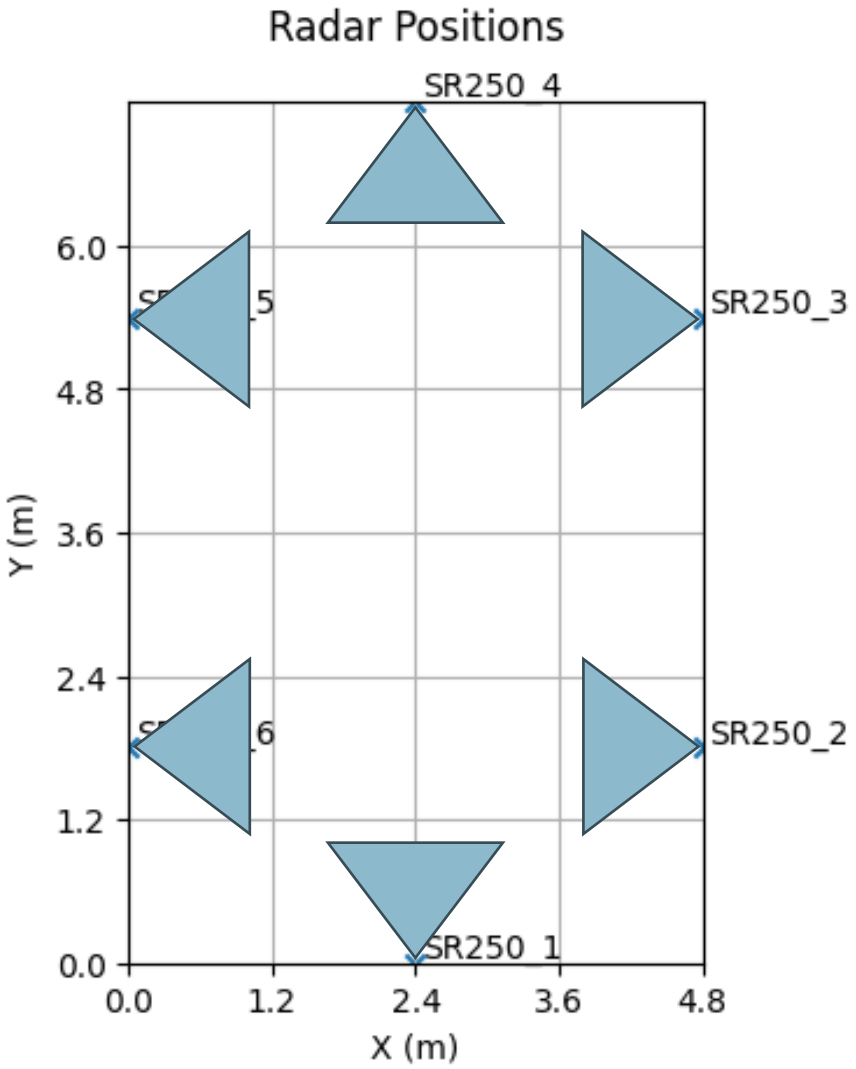
Dataset

Multi-Radar Multi-Antenna CIR Dataset

- **6 fixed UWB radar sensors,** positioned around the room perimeter
- Each radar has **3 antennas**, **120 range bins**, sampled at **25 Hz**
- Room dimensions: **4.8 m × 7.2 m**
- Radars positioned at **1.2 m height** at fixed positions around the room
- **24 acquisition windows**, each 300 seconds (7 500 frames)
- Total: **120 minutes** of data (180 000 frames)

Room Setup & Sensor Configuration

- **Radar model:** TSRR250 Ultra-Wideband radar
- **Number of radars:** 6, placed at fixed positions around the room perimeter
- **Antennas per radar:** 3 (providing angular diversity)
- **Room size:** 4.8 m × 7.2 m



Data Format

Each acquisition window is stored as a **`.npz`** file containing four arrays:

- **`radar_cir_iq`** shape `(T, 6, 3, 120, 2)`

  Complex CIR (I/Q) - T frames × 6 radars × 3 antennas × 120 bins × 2 (I, Q)

- **`people_xy`** shape `(T, 4, 2)`

  2D ground-truth positions in metres (x, y) for up to 4 people

- **`people_mask`** shape `(T, 4)`

  Boolean mask - indicates valid entries (zero-padded when fewer than 4 people)

- **`timestamps`** shape `(T,)`

  Acquisition timestamps for each frame

Acquisition Scenarios

24 acquisition windows with varying conditions:

- **0 to 4 subjects** per window
- Combinations of **random walking** and **seated** subjects
- **Serpentine walks** (structured path traversals)

Why this matters

- The diversity of scenarios tests generalisation across occupancy levels and movement patterns
- Your model must handle the empty room (0 people) as well as the full room (4 people)

Acquisition Setup

Dataset Access & Submission

How to Access the Dataset - Hugging Face

The dataset is published on **Hugging Face** (private organisation).

Step 1 - Create a Hugging Face account

- If you don't already have one, register at huggingface.co

Step 2 - Join the HAEEAI organization

- https://huggingface.co/organizations/HAEEAI/share/BWZUHibZiaCeOKdfJNlFEXHHwvVFdlUnfR

Step 3 - Download the dataset

- https://huggingface.co/datasets/HAEEAI/multi-person-localization

How to Access the Repository - GitHub Classroom

Your code and deliverables will be submitted via **GitHub Classroom**.

Accept the GitHub Classroom assignment

- Click the invitation link to create your team repository
- Form groups of **2 students**
- Invitation link: **[TO BE ANNOUNCED]**

What goes in the repository

- `submission/` -> code.py + model.tflite
- `report/` -> report.pdf
- `other_files/` -> training notebooks, experiments

Task Definition

Input & Output Specification

Input

- A single **`.npy`** file of shape `(T, 6, 3, 120, 2)`
- Contains raw radar CIR measurements (no ground truth)

Output

- A **`.jsonl`** file with one JSON object per frame
- Each object contains: `frame` (index) + `localizations` (list of [x, y] in metres)
- Maximum **4 people** per frame, minimum **0**
- Empty list [ ] if no person detected

Output Format - Example

Each line of the **`.jsonl`** output file is a valid JSON object:

```
{"frame": 36, "localizations": [[1.2, 3.4], [2.5, 5.1]]}
{"frame": 37, "localizations": [[1.3, 3.5]]}
{"frame": 38, "localizations": []}
```

Key fields

- `frame`: original frame index (0-based).
- `localizations`: list of [x, y] positions in meters, empty list if no person.

Coordinate System & Radar Selection

Coordinate System

- Origin at the **bottom-left corner** of the room
- **x** axis: positive to the **right**
- **y** axis: positive **upward**
- All positions are reported in **meters**

Radar Selection - Your Choice

- You are **free to use any subset or combination** of the 6 radars
- Trade-off: more radars = richer signal, but larger model input
- This is a design choice, experiment and justify in your report

Repository Structure

Required Repository Structure

```
submission/
        code.py                         ← inference script (required, exact name)
        model.tflite                    ← trained TFLite model (required, exact name)
report/
        report.pdf                      ← project report
other_files/
        ...                             ← training notebooks, scripts, experiments
evaluation/
        evaluate_constraint.py          ← ESP32-S3 constraint checker
        evaluate_performance.py         ← performance evaluator
        example/
                input_test.npy          ← example input
                output_test.jsonl       ← expected output
```

Inference Script — code.py

How it is called

```
python submission/code.py
    --input-path <path/to/input.npy>
    --output-path <path/to/output.jsonl>
```

Constraints

- `code.py` must not rely on packages beyond NumPy, SciPy, and TensorFlow
- Model must be in **TFLite format**
- File name must be exactly **`code.py`** and **`model.tflite`**

Deployment Constraints

ESP32-S3 Deployment Constraints

Your model will **NOT** be deployed on a real microcontroller. However, we will verify that it **meets the hardware constraints** of an ESP32-S3:

Check	Limit	Notes
Model file size	< 800 KB	Flash budget
Tensor arena (SRAM)	< 400 KB	No external PSRAM assumed
Quantization	INT8 recommended	4× smaller, faster on ESP32-S3
Forbidden operations	No LSTM / GRU / RNN	See note below

⚠️ **Important:** *A model containing LSTM or GRU layers can be converted to TFLite successfully — the conversion will not fail. However, TensorFlow Lite for Microcontrollers (TFLM), which runs on the ESP32-S3, supports only a limited subset of operations. LSTM, GRU, and RNN variants are **not supported** and will cause deployment to fail.*
Prefer standard layers: `Conv2D, DepthwiseConv2D, Dense, BatchNormalization, MaxPool2D, AveragePool2D, Add, ReLU`.
Full list of supported ops: `tensorflow/tflite-micro – micro_mutable_op_resolver.h`

Running the Constraint Check

A provided script checks whether your model meets ESP32-S3 constraints:

```
python evaluation/evaluate_constraint.py
    --model-path submission/model.tflite
```

What it checks

- Model file size < 800 KB
- Tensor arena (SRAM) < 400 KB
- Forbidden operations (LSTM / GRU / RNN / ...)

⚠ Run this before submission to avoid rejection!

Local Testing

Test Your Pipeline Locally (1/2)

Before submitting, test your full pipeline using the provided example:

1. Run inference

```
python submission/code.py \
    --input-path evaluation/example/input_test.npy \
    --output-path evaluation/example/my_output.jsonl
```

Note: the example input/output is derived from a portion of the training data. It is provided for format verification and sanity checking.

Test Your Pipeline Locally (2/2)

2. Check performance

```
python evaluation/evaluate_performance.py \
    --gt-path evaluation/example/output_test.jsonl \
    --pred-path evaluation/example/my_output.jsonl
```

3. Check ESP32-S3 constraints

```
python evaluation/evaluate_constraint.py \
    --model-path submission/model.tflite
```

⚠ Run both checks before every submission!

Evaluation & Scoring

Evaluation - Key Message

The final evaluation will be performed on a

different acquisition session

not seen during training.

Build a model that **generalises** - overfitting the training data will not help.

Grading

Grading

The project is worth **12 points out of 30**, divided as follows:

- **3 pts** - Baseline Accuracy
- **3 pts** - Edge Optimization
- **2 pts** - Documentation
- **2 pts** - Code and reproducibility
- **2 pts** - Innovation/Ranking

Submission deadlines:

- **July 15** - June/July session
- **September 1** - September session

Summary & Next Steps

What You Need to Do

1. Join the **Hugging Face** organisation and **download the dataset**
2. Accept the **GitHub Classroom assignment** (link TBA) and **form groups of 2**
3. Develop your **preprocessing, model training, and inference pipeline**
4. Convert model to **TFLite** and verify **ESP32-S3 constraints**
5. Test locally with the **provided example**
6. Submit **`code.py`**, **`model.tflite`**, **`report.pdf`**, and all project code

Document your choices and justify them in the report.

Good luck!

Questions & Answers

Asked Questions

Q1: Can the written exam and the project be in different sessions?

- **Yes**, as long as both are completed within the **same academic year**.

Q2: Can we use pre-existing models (e.g. from Hugging Face)?

- **Yes**, but the model **must respect all project constraints**

Q3: Can we form groups of 3 students?

- **No.** Groups must be of **exactly 2 students**, as stated in the project requirements.

Q4: What does "Innovation/Ranking" mean in the grading?

- It rewards **creative approaches** and **competitive accuracy**. Groups are **ranked**: higher-ranked, innovative solutions earn more points.